39.1 多 Agent 设计

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~290k tokens，输出 ~5k tokens（本节）

现代 AI 助手的能力边界正在迅速扩展——从简单的问答，到编码、浏览器自动化、日程管理、多模态理解。单一 Agent 难以在所有领域都表现最优。OpenClaw 的解决方案是**多 Agent 架构**（Multi-Agent Architecture）：允许用户在同一个 Gateway 实例上定义多个 Agent，每个 Agent 拥有独立的身份、工作空间、模型配置和技能集合。

> **衍生解释：Agent**
> 在 AI 领域，"Agent"指具有自主决策能力的软件实体。它能感知环境（接收消息、读取文件）、制定计划、调用工具执行动作，并根据反馈调整行为。与传统的"聊天机器人"不同，Agent 强调**自主性**和**工具使用能力**。本书统一使用英文原词"Agent"。

39.1.1 Agent 范围解析

多 Agent 系统的核心问题是：如何从配置中解析出所有 Agent、确定默认 Agent、以及将会话路由到正确的 Agent。 `agent-scope.ts` 是这一机制的核心实现。

Agent 列表解析

```ts
// src/agents/agent-scope.ts（简化）

function listAgents(cfg: OpenClawConfig): AgentEntry[] {
  const list = cfg.agents?.list;
  if (!Array.isArray(list)) return [];
  return list.filter(
    (entry): entry is AgentEntry =>
      Boolean(entry && typeof entry === "object")
  );
}

export function listAgentIds(cfg: OpenClawConfig): string[] {
  const agents = listAgents(cfg);
  if (agents.length === 0) return [DEFAULT_AGENT_ID];
  const seen = new Set<string>();
  const ids: string[] = [];
  for (const entry of agents) {
    const id = normalizeAgentId(entry?.id);
    if (seen.has(id)) continue;
    seen.add(id);
    ids.push(id);
  }
  return ids.length > 0 ? ids : [DEFAULT_AGENT_ID];
}
```

关键设计决策：

决策	实现	原因
空列表回退	返回 `[DEFAULT_AGENT_ID]`	零配置即可运行，单 Agent 场景无需配置
ID 去重	`Set` 过滤重复 ID	防止配置错误导致同一 Agent 注册多次
ID 标准化	`normalizeAgentId()` 统一大小写	`"Coder"` 和 `"coder"` 视为同一 Agent

`DEFAULT_AGENT_ID` 来自 `src/routing/session-key.ts`，是系统内置的默认标识符。当用户未配置任何 Agent 时，所有会话都路由到这个默认 Agent。

默认 Agent 选举

当配置了多个 Agent 时，系统需要确定哪个是"默认"Agent ——即没有显式指定 Agent 时使用的那个：

```typescript
export function resolveDefaultAgentId(cfg: OpenClawConfig): string {
  const agents = listAgents(cfg);
  if (agents.length === 0) return DEFAULT_AGENT_ID;

  const defaults = agents.filter((agent) => agent?.default);
  if (defaults.length > 1 && !defaultAgentWarned) {
    defaultAgentWarned = true;
    console.warn(
      "Multiple agents marked default=true; using the first."
    );
  }
  const chosen = (defaults[0] ?? agents[0])?.id?.trim();
  return normalizeAgentId(chosen || DEFAULT_AGENT_ID);
}
```

选举逻辑遵循**优先级链**：

1. 标记了 `default: true` 的 Agent（多个则取第一个，并发出警告）
2. 配置列表中的第一个 Agent
3. 系统内置 `DEFAULT_AGENT_ID`

`defaultAgentWarned` 是一个模块级布尔值，确保警告只输出一次，避免日志污染。

会话-Agent 绑定

每个会话通过 **Session Key** 绑定到特定 Agent。Session Key 的格式编码了 Agent 信息：
每个会话通过 **Session Key** 绑定到特定 Agent。Session Key 的格式编码了 Agent 信息：

```
export function resolveSessionAgentIds(params: {
  sessionKey?: string;
  config?: OpenClawConfig;
}): { defaultAgentId: string; sessionAgentId: string } {
  const defaultAgentId = resolveDefaultAgentId(params.config ?? {});
  const normalizedSessionKey = params.sessionKey?.trim().toLowerCase();
  const parsed = normalizedSessionKey
    ? parseAgentSessionKey(normalizedSessionKey)
    : null;
  const sessionAgentId = parsed?.agentId
    ? normalizeAgentId(parsed.agentId)
    : defaultAgentId;
  return { defaultAgentId, sessionAgentId };
}
```

`parseAgentSessionKey()` 从会话键中提取 Agent ID。例如，键 `agent:coder:subagent:abc123` 会被解析出 `agentId = "coder"` 。如果会话键不包含 Agent 前缀（如普通的 `"main"` 会话），则回退到默认 Agent。

ResolvedAgentConfig 类型

每个 Agent 的完整配置通过 `resolveAgentConfig()` 解析。返回类型 `ResolvedAgentConfig` 包含了 Agent 的所有可配置维度：

```
type ResolvedAgentConfig = {
  name?: string;            // 显示名称
  workspace?: string;       // 工作目录路径
  agentDir?: string;        // Agent 状态目录
  model?: AgentEntry["model"];      // 模型配置（字符串或 {primary, fallbacks}）
  skills?: AgentEntry["skills"];    // 技能过滤列表
  memorySearch?: ...;       // 记忆搜索配置
  humanDelay?: ...;         // 模拟人类打字延迟
  heartbeat?: ...;          // 心跳间隔
  identity?: ...;           // 身份/人格定义
  groupChat?: ...;          // 群聊配置
  subagents?: ...;          // 子 Agent 策略
  sandbox?: ...;            // 沙箱隔离配置
  tools?: ...;              // 工具白名单/黑名单
};
```

每个字段都经过严格的类型守卫，确保无效配置不会传递到下游：

```
model:
  typeof entry.model === "string" ||
  (entry.model && typeof entry.model === "object")
    ? entry.model
    : undefined,
```

这种"有效则取、无效则 undefined"的模式贯穿整个配置解析层。

模型解析：主模型与回退链

Agent 的模型配置支持两种格式：

```
// 简单格式：直接指定模型名
{ "model": "claude-opus-4-6" }

// 高级格式：主模型 + 回退链
{
  "model": {
    "primary": "claude-opus-4-6",
    "fallbacks": ["claude-sonnet-4-20250514", "gpt-4o"]
  }
}
```

对应的解析函数：

```
// 主模型
export function resolveAgentModelPrimary(
  cfg: OpenClawConfig,
  agentId: string,
): string | undefined {
  const raw = resolveAgentConfig(cfg, agentId)?.model;
  if (typeof raw === "string") return raw.trim() || undefined;
  return raw?.primary?.trim() || undefined;
}

// 回退链（注意：空数组 [] 是有效覆盖，表示禁用全局回退）
export function resolveAgentModelFallbacksOverride(
  cfg: OpenClawConfig,
  agentId: string,
): string[] | undefined {
  const raw = resolveAgentConfig(cfg, agentId)?.model;
  if (!raw || typeof raw === "string") return undefined;
  if (!Object.hasOwn(raw, "fallbacks")) return undefined;
  return Array.isArray(raw.fallbacks) ? raw.fallbacks : undefined;
}
```

`resolveAgentModelFallbacksOverride` 的一个精妙之处在于：它使用 `Object.hasOwn` 来区分"未设置 fallbacks"和"显式设置 fallbacks 为空数组"。后者意味着用户**故意禁用**全局回退模型链——这是一个重要的语义区分。

39.1.2 Agent 路径系统

每个 Agent 需要两个独立的目录：**工作空间**（workspace）和 **Agent 状态目录**（agent dir）。

工作空间隔离

```
// src/agents/agent-scope.ts
export function resolveAgentWorkspaceDir(
  cfg: OpenClawConfig,
  agentId: string,
) {
  const id = normalizeAgentId(agentId);
  // 1. Agent 自身配置的 workspace
  const configured = resolveAgentConfig(cfg, id)?.workspace?.trim();
  if (configured) return resolveUserPath(configured);

  // 2. 默认 Agent 使用全局 defaults.workspace 或内置路径
  const defaultAgentId = resolveDefaultAgentId(cfg);
  if (id === defaultAgentId) {
    const fallback = cfg.agents?.defaults?.workspace?.trim();
    if (fallback) return resolveUserPath(fallback);
    return DEFAULT_AGENT_WORKSPACE_DIR;
  }

  // 3. 非默认 Agent 使用 ~/.openclaw/workspace-{id}
  return path.join(os.homedir(), ".openclaw", `workspace-${id}`);
}
```

路径解析遵循**三级回退**策略：

```
Agent 自身配置
    ↓ (未设置)
默认 Agent? → 全局 defaults.workspace → DEFAULT_AGENT_WORKSPACE_DIR
非默认 Agent? → ~/.openclaw/workspace-{id}
```

非默认 Agent 的工作空间自动按 ID 后缀隔离。例如， `coder` Agent 的工作空间为 `~/.openclaw/workspace-coder` ， `researcher` Agent 的工作空间为 `~/.openclaw/workspace-researcher` 。这确保了不同 Agent 的文件操作互不干扰。

Agent 状态目录

Agent 状态目录存放 Agent 的持久化状态数据（如记忆文件、SOUL 配置等）：

```ts
// src/agents/agent-scope.ts
export function resolveAgentDir(
  cfg: OpenClawConfig,
  agentId: string,
) {
  const id = normalizeAgentId(agentId);
  const configured = resolveAgentConfig(cfg, id)?.agentDir?.trim();
  if (configured) return resolveUserPath(configured);
  const root = resolveStateDir(process.env, os.homedir);
  return path.join(root, "agents", id, "agent");
}
```

默认路径为 `~/.openclaw/agents/{id}/agent` 。 `resolveStateDir()` 会检查环境变量（如 `OPENCLAW_STATE_DIR` ）来确定状态根目录。

`agent-paths.ts` 提供了一个更简单的入口，专门用于解析当前进程的 Agent 目录：

```ts
// src/agents/agent-paths.ts
export function resolveOpenClawAgentDir(): string {
  const override =
    process.env.OPENCLAW_AGENT_DIR?.trim() ||
    process.env.PI_CODING_AGENT_DIR?.trim();
  if (override) return resolveUserPath(override);
  return resolveUserPath(
    path.join(resolveStateDir(), "agents", DEFAULT_AGENT_ID, "agent")
  );
}

export function ensureOpenClawAgentEnv(): string {
  const dir = resolveOpenClawAgentDir();
  if (!process.env.OPENCLAW_AGENT_DIR) {
    process.env.OPENCLAW_AGENT_DIR = dir;
  }
  if (!process.env.PI_CODING_AGENT_DIR) {
    process.env.PI_CODING_AGENT_DIR = dir;
  }
  return dir;
}
```

`ensureOpenClawAgentEnv()` 做了双向写入（ `OPENCLAW_AGENT_DIR` 和 `PI_CODING_AGENT_DIR` ），这是为了兼容旧版本的环境变量命名。

39.1.3 Agent 默认配置

`defaults.ts` 定义了 Agent 的全局默认值：

```
// src/agents/defaults.ts
export const DEFAULT_PROVIDER = "anthropic";
export const DEFAULT_MODEL = "claude-opus-4-6";
export const DEFAULT_CONTEXT_TOKENS = 200_000;
```

这些常量作为所有 Agent 配置的**最终兜底**——当用户未指定模型提供商、模型名称或上下文窗口大小时使用。

常量	值	含义
`DEFAULT_PROVIDER`	`"anthropic"`	默认使用 Anthropic 作为 LLM 提供商
`DEFAULT_MODEL`	`"claude-opus-4-6"`	默认使用 Claude Opus 4.6 模型
`DEFAULT_CONTEXT_TOKENS`	`200_000`	上下文窗口 200K tokens

这些默认值反映了 OpenClaw 的核心定位：以 Anthropic 的 Claude 模型为主力，同时通过配置支持其他提供商（如 OpenAI、Google Gemini 等）。 `DEFAULT_CONTEXT_TOKENS = 200_000` 是一个保守估计——当模型元数据不可用时（例如自部署的开源模型），系统使用此值来管理上下文窗口。

39.1.4 多 Agent 配置实践

一个典型的多 Agent 配置示例：

```
// ~/.openclaw/openclaw.json
{
  "agents": {
    "list": [
      {
        "id": "main",
        "name": "主助手",
        "default": true,
        "model": "claude-opus-4-6",
        "skills": ["coding-agent", "github", "weather"],
        "subagents": {
          "allowAgents": ["researcher", "coder"]
        }
      },
      {
        "id": "researcher",
        "name": "研究员",
        "model": {
          "primary": "claude-sonnet-4-20250514",
          "fallbacks": []
        },
        "skills": ["web-search"],
        "workspace": "~/research-workspace"
      },
      {
        "id": "coder",
        "name": "编码助手",
        "model": "claude-opus-4-6",
        "skills": ["coding-agent", "github"],
        "sandbox": {
          "mode": "auto",
          "scope": "session"
        }
      }
    ],
    "defaults": {
      "workspace": "~/openclaw-workspace"
    }
  }
}
```

在这个配置中：

- **main** 是默认 Agent，拥有生成子 Agent 的权限（可以派生 `researcher` 和 `coder`）
- **researcher** 使用较经济的 Sonnet 模型，显式禁用回退链（`fallbacks: []`），拥有独立的工作空间
- **coder** 启用了沙箱隔离（`sandbox.mode: "auto"`），以 session 为粒度创建独立容器

三个 Agent 的目录布局如下：

```
~/.openclaw/
├── agents/
│   ├── main/agent/          ← main 的状态目录
│   ├── researcher/agent/    ← researcher 的状态目录
│   └── coder/agent/         ← coder 的状态目录
├── workspace/               ← main（默认 Agent）的工作空间
├── workspace-coder/         ← coder 的工作空间
└── ...

~/research-workspace/        ← researcher 的自定义工作空间
```

本节小结

1. **Agent 列表解析**：`listAgentIds()` 从 `agents.list` 配置中提取去重后的 Agent ID 列表，空配置时回退到系统默认 Agent。
2. **默认 Agent 选举**：优先选择标记 `default: true` 的 Agent，否则取列表首项，最终回退到 `DEFAULT_AGENT_ID`。
3. **会话路由**：通过 `parseAgentSessionKey()` 从 Session Key 中提取目标 Agent ID，实现会话到 Agent 的精准绑定。
4. **Agent 配置解析**：`ResolvedAgentConfig` 涵盖 11 个维度（模型、技能、工作空间、沙箱等），每个字段都有严格的类型守卫。
5. **模型回退链**：支持简单字符串和 `{primary, fallbacks}` 两种格式，空数组 `[]` 用于显式禁用全局回退。
6. **路径隔离**：每个 Agent 拥有独立的工作空间和状态目录，默认 Agent 使用全局路径，非默认 Agent 按 ID 后缀隔离。
7. **默认常量**：`DEFAULT_PROVIDER="anthropic"`、`DEFAULT_MODEL="claude-opus-4-6"`、`DEFAULT_CONTEXT_TOKENS=200_000` 作为最终兜底配置。